Puppy Raffles Audit Report

Version 1.0

alva-p.xyz

March 19, 2026

- Prepared by: Cyfrin
- Lead Auditors: Alvaro Pineda

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the enterRaffle function with the following parameters: address[] participants: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & value if they call the refund function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a feeAddress to take a cut of the value, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Table 1: Likelihood

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: df177a5c98089fb13f35bc0edff22056d59d4f93

Scope

```
./src/
 PuppyRaffle.sol
```

Roles

- Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the changeFeeAddress function.

- Player - Participant of the raffle, has the power to enter the raffle with the enterRaffle function and refund value through refund function.

Issues found

Table 2: Issues found summary

Severity	Number of issues found
High	3
Medium	2
Low	1
Info/Gas	9
Total	16

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description The `PuppyRaffle::refund` function does not allow follow CEI (checks, effects, interactions) and as result, enables participants to drain the contract balance. In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that external call do we update the `PuppyRaffle::players` array.

```
 function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the
            player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player
            already refunded, or is not active");

@>      payable(msg.sender).sendValue(entranceFee);
@>      players[playerIndex] = address(0);

        emit RaffleRefunded(playerAddress);
    }
```

A player who has entered the raffle could have `fallback`/`receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained.

Impact All fees paid by raffle entrants could be stolen by a malicious participant.

Proof of concept

1. User enters the raffle
2. Attackers sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attackers enters the raffle
4. Atacckers calls `PuppyRaffle::refund` from their attack contract, draining the contract balancer

Proof of code

```
function test_reetrancyRefund() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
            puppyRaffle);
        address attackUser = makeAddr("attackUser");
        vm.deal(attackUser, 1 ether);

        uint256 startingAttackContractBalance = address(
            attackerContract).balance;
        uint256 startingContractBalance = address(puppyRaffle).balance;

        // attack
        vm.prank(attackUser);
        attackerContract.attack{value: entranceFee}();

        emit log_named_uint("starting attacker contract balance",
            startingAttackContractBalance);
        emit log_named_uint("starting contract balance",
            startingContractBalance);
        emit log_named_uint("ending attacker contract balance", address
            (attackerContract).balance);
        emit log_named_uint("ending contract balance", address(
            puppyRaffle).balance);
    }
```

Recommended Mitigation To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally, we should move the event

emission up as well.

```
function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

+       players[playerIndex] = address(0);
+       emit RaffleRefunded(playerAddress);

        payable(msg.sender).sendValue(entranceFee);

-       players[playerIndex] = address(0);
-       emit RaffleRefunded(playerAddress);
    }
```

[H-2] Weak randomness is `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winner puppy.

Description Hashing `msg.sender`, `block.timestamp`, and `block.difficulty` together creates a predictable find number. A predictable number is not a good random number. Malicious users can manipulate thee values or know them ahead of time to choose the winner of the raffle themselvles.

Impact Any user can influence the winner on the raffle, winning the money and selecting rarest puppys. Making the entire raffle worthless if it become a gas war as to who win the raffles.

Proof of Concept 1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use to predict when/how to participate. `block.difficulty` was recently replaced with prevrandao. 2. User can mine/manipulate the `msg.sender` value to result in thir address being used to generated the winner! 3. Users can revert their `selectWinner` transaction if they don´t like the winner or the resulting puppy.

Using on-chain values as a randomness see is a [well-documented attack vector]

Recommend Mitigation Consider using a cryptographycally provable random number generator such as Chainlink VRF.

[H-3] Integrer overflow of `PuppyRaffle::totalFees` loses fees

Description In solidity versions prior to `0.8.0` integrers were subject to integrer overflows.

```
uint64 myVar = type(uint64).max
myVar = myVar + 1
```

Impact In `PuppyRaffle::selectWinner`, `totalFees` are accuulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract

Proof of Concept

1. We conclude a raffle of 4 players
2. We then have 89 players enter a new raffle, and conclude the raffle
3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);

totalFees = 800000000000000000 + 17800000000000000000000

//and this overflow!
```

4. You shold not be able to withdraw, due to the line in `PuppyRaffle::withdrawFees`:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
        uint256 feesToWithdraw = totalFees;
```

Altought you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intenden design of the protocol. At some point, there will be too much `balance` in the contract that was above `require` will be imposible to hit.

Recommend Mitigation 1. Use a new version of solidity and a `uint256` instead of `uint64` for `PuppyRaffles::totalFees` 2. You could also use the `SafeMath` library of OpenZeppelin for version 0.7.6 of solidity, however you would still have hard time with `uint64` type if too many fees are collected. 3. Remove the balance check from `PuppyRaffles::withdrawFees`

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

Medium

[M-1] Looping through players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential danial of service (DoS) attack, incrementing gas cost for future entrants

IMPACT: MEDIUM LIKELIHOOD: MEDIUM

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check duplicates. However the longer the `PuppyRaffle::enterRaffle` array is, the more checks

a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array, is an additional check the loop will have to make.

```
// @Audit DoS
        for (uint256 i = 0; i < players.length - 1; i++) { // ? - can we do this in a more gas efficient way?
            for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle: Duplicate player"); // denied service bug
            }
```

Impact: The gas costs for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering, and causing a rush at the start of the raffle to be one of the first entrants in queue.

An attacker might make the `PuppyRaffle::entrants` array so big, that no one else enters, guaranteeing themselves to win.

Proof of Concept:

If we have two sets of 100 players enter, the gast costs will be a such: - 1st 100 players: 6503225 - 2nd 100 players: 42160926

There is a big difference costs of gas.

PoC

Place the following test into `PuppyRaffleTest.t.sol`.

```
function test_denialOfService() public {
        //address[] memory players = new address[](1);
        //players[0] = playerOne;
        //puppyRaffle.enterRaffle{value: entranceFee}(players);
        //assertEq(puppyRaffle.players(0), playerOne);
        // Agreguemos 100 usuarios
        uint256 playersNumberFirst = 100;
        address[] memory players = new address[](playersNumberFirst);
        for (uint256 i = 0; i < playersNumberFirst; i++) {
            players[i] = address(i);
            // player 1
            // player 2 ... etc
        }
        uint256 gasStart = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(players);
        uint256 gasEnd = gasleft();

        uint256 gasUsedFirst = gasStart - gasEnd;
```

```
        emit log_named_uint("Gas used to enter raffle with 100 players", gasUsedFirst);

        // Now for the 200 players
        uint256 playersNumberSecond = 200;
        address[] memory playersTwo = new address[](playersNumberSecond);
        for (uint256 i = 0; i < playersNumberSecond; i++) {
            playersTwo[i] = address(i + playersNumberSecond);
            // player 1
            // player 2 ... etc
        }
        uint256 gasStartSecond = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * playersTwo.length}(playersTwo);
        uint256 gasEndSecond = gasleft();
        uint256 gasUsedSecond = gasStartSecond - gasEndSecond;
        emit log_named_uint("Gas used to enter raffle with 200 players", gasUsedSecond);

        assert(gasUsedFirst < gasUsedSecond);

    }
```

Recommended Mitigation: There a few recomendations.

1. Consider allowing duplicates. Users can make new wallets addresses anyways, so duplicate check doesn't prevent the same person from entering multiple times, only the same wallet addresses.
2. Consider using a mapping for check for duplicates. This would allow constant time lookup of whatever a user already entered.

[M-2] Smart contract wallets raffle winners without a `receive` or a `fallback` function will block the start of a new contest

Description The `PuppyRaffle::selectWinner` function is responsible for ressetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Users could easily call the `selectWInner` function again and non-wallet entrants could enter, but it could cost a lot due to the duplicate check and a lottery reset could get very challenging.

Impact The `PuppyRaffle:selectWinner` function could revert many times, making a lottery reset difficult.

Proof of Concepts

1. 10 smarts contracts wallets enter to the lottery without a fallback or receive function.
2. The lottery ends.
3. The `selectWinner` function would't work, even though the lottery is over!

Recommended mitigation There a few options to mitigate this issue. 1. Do not allow smart contracts wallets entrants. 2. Create mapping of addresses -> payour so winners can pull their funds out themselves with a new `claimPrize` function, putting the owness on the winner to claim ther prize. (Recommended) 3.

Low

[L-1] `PuppyRaffle::getActivePlayersIndex` returns to 0 for no-existent players and for players at index 0, causing a player at index 0 to incorrectly think they have no entered the raffle.

Description If the player is in the `PuppyRaffle::players` array in the index 0, this will return 0. But according to the natspec, it will also return 0 if the player is not in the array.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
        for (uint256 i = 0; i < players.length; i++) {
            if (players[i] == player) {
                return i;
            }
        }
        return 0;
    }
```

Impact A plyaer at the index 0 may incorrectly think they have no entered to the raffle, and attempt to enter the raffle again, wasting gas.

Proof of concept

1. Users enter the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayersIndex` returns 0
3. User think they have not entered correctly due the function documentation.

Recommended mitigation The easist recommendation would be revert if the player is not in the array insted of returning 0.

You could also reserve the 0th position for any competition. A better solution might be to return a `int256` where the function returns -1 if the player is not active.

Gas

[G-1] Unchanged state variables should be declared constan or inmutable

Reading for storage is more expensive than reading from a constant or inmutable variable

Instances:
- `PuppyRaffle::raffleDurations` should be `inmutable` - `PuppyRaffle::commonImageUri` should be `constant` - `PuppyRaffle::rareImageUri` should be `constant` - `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage variables in a loop should be cached

Everytime you call `players.lenght` you read from storage, as opposed to memory which is more gas efficient.

```
for (uint256 i = 0; i < players.length - 1; i++) { // ? - can we do this in a more gas efficient way?
            for (uint256 j = i + 1; j < players.length; j++) {
                require(players[i] != players[j], "PuppyRaffle: Duplicate player");
            }
        }
        emit RaffleEnter(newPlayers);
```

Information

[I-1] Solidity pragma should be specific, not wide

Consider using a specific version of Solidity, in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0`, use `pragma solidity 0.8.0`

- Found in src/PuppyRaffle.sol

[I-2] Using an outdated version of Solidity is not reccomended

Solc frequently releases a new compiler versions. Using a old version prevents access to new Solidity security checks. We also recommend avoiding compelx pragma statement.

Recommendations

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

For more information : https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

[I-3] Missing checks for `address(0)` when assigning values to address stats variables

Assigning values to address state variables without checking for `address(0)`

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practices

It's best to keep code clean and follow CEI (checks, effects, interactions).

```
-        (bool success,) = winner.call{value: prizePool}("");
-        require(success, "PuppyRaffle: Failed to send prize pool to winner");
         _safeMint(winner, tokenId);
+        (bool success,) = winner.call{value:prizePool}("");
+        require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-5] Use of "magic" number is discouraged

It can be confusing to see number literals in a codebase, and it's much more readeable if the numbers are given a name.

Examples:

```
    uint256 prizePool = (totalAmountCollected * 80) / 100;
    uint256 fee = (totalAmountCollected * 20) / 100;
```

Instead you could:

```
// uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
// uint256 public constant FEE_PERCENTAGE = 20;
// uint256 public constant POOL_PRECISION = 100;
```

[I-6] State changes are missing events

[I-7] `PuppyRaffles::_isActivePlayers` is never used and should be removed